Глава 3. STRIDE

Как вы узнали в главе 1 «Dive In and Threat Model!», STRIDE — это аббревиатура, которая расшифровывается как Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service и Elevation of Privilege. Подход к моделированию угроз STRIDE был изобретён Лореном Конфелдером и Праэритом Гаргом (Kohnfelder, 1999). Эта структура и мнемоника были разработаны, чтобы помочь людям, разрабатывающим ПО, определить типы атак, которым ПО обычно подвергается.

Метод или методы, которые вы используете для обдумывания угроз, имеют много разных названий: поиск угроз, перечисление угроз, анализ угроз, выявление угроз, обнаружение угроз. Каждое из них подразумевает немного不同的 оттенок подхода. Существуют ли угрозы в ПО или на диаграмме? Тогда вы их находите. Существуют ли они в умах людей, выполняющих анализ? Тогда вы делаете анализ или выявление. Ни одно описание не выделяется как всегда или явно предпочтительное, но эта книга обычно говорит о поиске угроз как о надмножестве всех этих идей. Использование STRIDE больше похоже на технику выявления, с ожиданием, что вы или ваша команда понимаете структуру и знаете, как её использовать. Если вы не знакомы со STRIDE, обширные таблицы и примеры предназначены, чтобы научить вас использовать её для обнаружения угроз.

Эта глава объясняет, что такое STRIDE и почему она полезна, включая разделы, охватывающие каждый компонент мнемоники STRIDE. Каждый раздел, посвящённый конкретной угрозе, предоставляет более глубокое объяснение угрозы, подробную таблицу примеров для этой угрозы, а затем обсуждение примеров. Таблицы и примеры предназначены, чтобы научить вас использовать STRIDE для обнаружения угроз. Вы также узнаете о подходах, построенных на STRIDE: STRIDE-per-element, STRIDE-per-interaction и DESIST. Другой подход, построенный на STRIDE, — игра Elevation of Privilege — рассматривается в главах 1 «Dive In and Threat Model!» и 12 «Requirements Cookbook», а также в Приложении C «Attacker Lists».

Понимание STRIDE и почему она полезна

Угрозы STRIDE — это противоположность некоторых свойств, которые вы хотели бы видеть в своей системе: аутентичность, целостность, неотказуемость, конфиденциальность, доступность и авторизация. Таблица 3-1 показывает угрозы STRIDE, соответствующее свойство, которое вы хотели бы поддерживать, определение, наиболее типичных жертв и примеры.

Таблица 3-1: Угрозы STRIDE

УГРОЗА	НАРУШАЕМОЕ СВОЙСТВО	ОПРЕДЕЛЕНИЕ УГРОЗЫ	ТИПИЧНЫЕ ЖЕРТВЫ	ПРИМЕРЫ
Spoofing (Подмена)	Аутентификация	Притворяться чем-то или кем-то, кем	Процессы, внешние	Ложное заявление о том, что вы —

УГРОЗА	НАРУШАЕМОЕ СВОЙСТВО	ОПРЕДЕЛЕНИЕ УГРОЗЫ	ТИПИЧНЫЕ ЖЕРТВЫ	ПРИМЕРЫ
		вы не являетесь	сущности, люди	Acme.com, winsock.dll, Барак Обама, полицейский или Нигерийская анти-мошенническая группа
Tampering (Несанкционированное изменение)	Целостность	Изменение чего-то на диске, в сети или в памяти	Хранилища данных, потоки данных, процессы	Изменение электронной таблицы, бинарного файла важной программы или содержимого базы данных на диске; изменение, добавление или удаление пакетов по сети, локальной или через Интернет, проводной или беспроводной; изменение либо данных, используем

УГРОЗА	НАРУШАЕМОЕ СВОЙСТВО	ОПРЕДЕЛЕНИЕ УГРОЗЫ	ТИПИЧНЫЕ ЖЕРТВЫ	ПРИМЕРЫ
				ых программой, либо самой работающей программы
Repudiation (Репудиация)	Неотказуемость	Заявление, что вы чего-то не делали или не несёте ответственности. Репудиация может быть честной или ложной, и ключевой вопрос для проектировщиков систем: какие у вас есть доказательства?	Процесс	Процесс или система: «Я не нажимал большую красную кнопку» или «Я не заказывал этот Ferrari». Обратите внимание, что репудиация несколько выбивается из ряда; она выходит за техническую природу других угроз на бизнес-уровень.
Information Disclosure (Раскрытие информации)	Конфиденциальность	Предоставление информации тому, кто не имеет права её видеть	Процессы, хранилища данных, потоки данных	Наиболее очевидный пример — разрешение доступа к файлам, электронной почте или базам данных, но

УГРОЗА	НАРУШАЕМОЕ СВОЙСТВО	ОПРЕДЕЛЕНИЕ УГРОЗЫ	ТИПИЧНЫЕ ЖЕРТВЫ	ПРИМЕРЫ
				раскрытие информации может также включать имена файлов («Termination for John Doe.docx»), пакеты в сети или содержимое памяти программы.
Denial of Service (Отказ в обслуживании)	Доступность	Поглощение ресурсов, необходимых для предоставления сервиса	Процессы, хранилища данных, потоки данных	Программа, которую можно обманом заставить использовать всю свою память; файл, заполняющий диск; или такое количество сетевых соединений, что реальный трафик не может пройти
Elevation of Privilege	Авторизация	Позволение кому-то делать то, на	Процесс	Позволение обычному пользовател

УГРОЗА	НАРУШАЕМОЕ СВОЙСТВО	ОПРЕДЕЛЕНИЕ УГРОЗЫ	ТИПИЧНЫЕ ЖЕРТВЫ	ПРИМЕРЫ
(Повышение привилегий)		что он не авторизован		ю выполнять код как admin; позволение удалённому человеку без каких-либо привилегий запускать код

В таблице 3-1 «типичные жертвы» — это те, кто наиболее вероятно пострадает: например, вы можете подменить программу, запустив программу с тем же именем, или поместив программу с этим именем на диск. Вы можете подменить конечную точку на той же машине, заняв или spliced порт. Вы можете подменить пользователей, захватив их аутентификационную информацию путём подмены сайта, предполагая, что они повторно используют учётные данные на разных сайтах, путём брутфорса (онлайн или офлайн) или путём повышения привилегий на их машине. Вы также можете изменить базу данных аутентификации и затем подменить с фальсифицированными учётными данными.

Обратите внимание, что, используя STRIDE для поиска угроз, вы просто перечисляете вещи, которые могут пойти не так. Точные механизмы того, как это может пойти не так, — это то, что вы можете разработать позже. (На практике это может быть легко или очень сложно. Могут быть защиты на месте, и если вы скажете, например, «Кто-то мог бы изменить таблицы управления», кто-то другой может сказать: «Нет, не мог бы, потому что...») Может быть полезно записать эти возможные атаки, потому что даже если есть мера защиты, эта мера — тестируемая функция, и вы должны убедиться, что у вас есть тест-кейс.

Вы иногда будете слышать, как STRIDE называют «категориями STRIDE» или «таксономией STRIDE». Эта формулировка не полезна, потому что STRIDE не задумывалась как таксономия и не является ею. Легко найти вещи, которые трудно категоризировать с помощью STRIDE. Например, ранее вы узнали об изменении базы данных аутентификации и затем подмене. Следует ли записать это как угрозу несанкционированного изменения или как угрозу подмены? Простой ответ: это не имеет значения. Если вы уже придумали атаку, зачем утруждать себя помещением её в категорию? Цель STRIDE — помочь вам находить атаки. Категоризация их может помочь вам определить правильные защиты или может быть пустой тратой усилий. Попытки использовать STRIDE для категоризации угроз могут быть разочаровывающими, и эти усилия заставляют некоторых людей отвергать STRIDE, но это немного похоже на выплёскивание ребёнка вместе с водой.

Угрозы подмены

Подмена — это притворство чем-то или кем-то, кем вы не являетесь. Таблица 3-1 включает примеры заявления о том, что вы — Acme.com, winsock.dll, Барак Обама или Нигерийская анти-мошенническая служба. Каждый из них — пример不同的 подкатегории подмены. Первый пример, притворство Acme.com (или Google.com и т.д.), влечёт подмену идентичности сущности по сети. Нет посреднической власти, которая берёт на себя ответственность за то, чтобы сказать вам, что Acme.com — это сайт, который я имею в виду, когда пишу эти слова. Это отличается от второго примера, поскольку Windows включает winsock.dll. Вы должны иметь возможность попросить операционную систему действовать как посредническая власть и привести вас к winsock. Если у вас есть свои собственные DLL, вам нужно убедиться, что вы открываете их с правильным путём ($installdir$ \dll); иначе кто-то может подставить одну в рабочем каталоге и заставить ваш код делать то, что они хотят. (Аналогичные проблемы существуют с Unix и LD_PATH.) Третий пример, подмена Барака Обамы, — это случай притворства конкретным человеком. Сравните это с четвёртым примером, притворством президентом Соединённых Штатов или Нигерийской анти-мошеннической службой. В этих случаях атакующий притворяется, что находится в роли. Эти угрозы подмены изложены в таблице 3-2.

Таблица 3-2: Угрозы подмены

УГРОЗА	ПРИМЕРЫ	ЧТО ДЕЛАЕТ АТАКУЮЩИЙ	ПРИМЕЧАНИЯ
Подмена процесса на той же машине	Создаёт файл до реального процесса. Переименование/связывание. Создание троянского «su» и изменение пути. Переименование. Название своего процесса «sshd»		
Подмена файла	Создаёт файл в локальном каталоге. Это может быть библиотека, исполняемый файл или конфигурационный файл. Создаёт ссылку и изменяет её. С точки зрения атакующего, изменение должно произойти между проверкой ссылки и доступом к ссылке. Создаёт множество файлов в ожидаемом каталоге. Автоматизация упрощает создание 10 000 файлов в /tmp, чтобы заполнить пространство файлов с		

УГРОЗА	ПРИМЕРЫ	ЧТО ДЕЛАЕТ АТАКУЮЩИЙ	ПРИМЕЧАНИЯ
	именами /tmp / "pid. NNNN" или похожими. ARP-спуфинг		
Подмена машины	IP-спуфинг. DNS-спуфинг. Прямой или обратный DNS. Компрометация TLD, регистратора или DNS-оператора		
Подмена человека	IP-перенаправление. На уровне коммутатора или маршрутизатора. Устанавливает отображаемое имя в электронной почте. Захватывает реальную учётную запись		
Подмена роли	Объявляет себя этой ролью. Иногда открывает специальную учётную запись с соответствующим именем		

Подмена процесса или файла на той же машине

Если атакующий создаёт файл до реального процесса, то если ваш код не осторожен при создании нового файла, атакующий может предоставить данные, которые ваш код интерпретирует, думая, что ваш код (или предыдущий экземпляр или поток) записал эти данные, и им можно доверять. Аналогично, если права доступа к файлу для канала, локального вызова процедур и т.д. управляются плохо, атакующий может создать эту конечную точку, запутав всё, что пытается её использовать позже.

Подмена процесса или файла на удалённой машине может работать либо путём создания поддельных файлов или процессов на ожидаемой машине (возможно, получив права администратора), либо путём притворства ожидаемой машиной, что рассматривается далее.

Подмена машины

Атакующие могут подменять удалённые машины на различных уровнях сетевого стека. Эти атаки подмены могут влиять на представление вашего кода о мире как клиента, сервера или пира. Они могут подменять ARP-запросы, если они локальны, они могут подменять IP-пакеты, чтобы создать впечатление, что они приходят откуда-то, откуда они не приходят, и они могут подменять DNS-пакеты. DNS-спуфинг может произойти, когда вы выполняете прямой или обратный поиск. Атакующий может подменить DNS-ответ на прямой запрос, который, как он ожидает, вы сделаете. Они также могут настроить DNS-записи для машин, которые они контролируют, так что когда ваш код выполняет

обратный поиск (преобразование IP в FQDN), их DNS-сервер возвращает имя в домене, который они не контролируют — например, заявляя, что 10.1.2.3 — это update.microsoft.com. Конечно, как только атакующие подменили машину, они могут либо подменить, либо действовать как человек посередине для процессов на этой машине. Варианты второго порядка этой угрозы включают кражу аутентификаторов машины, таких как криптографические ключи, и их использование как части атаки подмены.

Атакующие также могут подменять на более высоких уровнях. Например, фишинговые атаки включают множество актов подмены. Обычно есть подмена электронной почты от «вашего» банка и подмена веб-сайта этого банка. Когда кто-то попадается на это письмо, кликает по ссылке и посещает банк, он вводит свои учётные данные, отправляя их на подменённый веб-сайт. Затем атакующий совершает последний акт подмены: он входит в ваш банковский счёт и переводит ваши деньги себе или сообщнику. (Это может быть один атакующий или набор атакующих, договаривающихся друг с другом об оказанных услугах.)

Подмена человека

Основные категории подмены людей включают доступ к учётной записи человека и притворство им через альтернативную учётную запись. Фишинг — распространённый способ получить доступ к чужой учётной записи. Однако часто мало что мешает любому создать учётную запись и притворяться вами. Например, атакующий может создать учётные записи на сайтах вроде LinkedIn, Twitter или Facebook и притворяться вами, Адамом Шостаком, который написал эту книгу, или богатым свергнутым принцем, пытающимся вывезти свои деньги из страны.

Угрозы несанкционированного изменения

Несанкционированное изменение — это модификация чего-то, обычно на диске, в сети или в памяти. Это может включать изменение данных в электронной таблице (с помощью программы вроде Excel или другого редактора), изменение бинарного файла или конфигурационного файла на диске или изменение более сложной структуры данных, такой как база данных на диске. В сети пакеты могут быть добавлены, изменены или удалены. Иногда проще добавить пакеты, чем редактировать их на лету, и программы на удивление плохо справляются с обработкой дополнительных копий данных безопасно. Больше примеров несанкционированного изменения — в таблице 3-3.

Таблица 3-3: Угрозы несанкционированного изменения

УГРОЗА	ПРИМЕРЫ	ЧТО ДЕЛАЕТ АТАКУЮЩИЙ	ПРИМЕЧАНИЯ
Несанкционированное изменение файла	Изменяет файл, которым владеет он и на который полагаетесь вы. Изменяет файл, которым владеете вы. Изменяет файл на		

УГРОЗА	ПРИМЕРЫ	ЧТО ДЕЛАЕТ АТАКУЮЩИЙ	ПРИМЕЧАНИЯ
	файловом сервере, которым владеете вы. Изменяет файл на своём файловом сервере. Много веселья, когда вы включаете файлы из удалённых доменов. Изменяет файл на своём файловом сервере. Замечали, как много XML включает удалённые схемы? Изменяет ссылки или перенаправления		
Несанкционированное изменение памяти	Изменяет ваш код. Трудно защититься, когда атакующий уже выполняет код как тот же пользователь		
Несанкционированное изменение сети	Изменяет данные, которые он предоставил вашему API. Передавайте по значению, а не по ссылке при пересечении границы доверия		
Несанкционированное изменение сети	Перенаправляет поток данных на свою машину. Часто этап 1 несанкционированного изменения. Изменяет данные, текущие по сети. Ещё проще и веселее, когда сеть беспроводная (WiFi, 3G		

УГРОЗА	ПРИМЕРЫ	ЧТО ДЕЛАЕТ АТАКУЮЩИЙ	ПРИМЕЧАНИЯ
	и т.д.). Усиливает атаки подмены		

Несанкционированное изменение файла

Атакующие могут изменять файлы везде, где у них есть разрешение на запись. Когда ваш код должен полагаться на файлы, которые другие могут записывать, есть вероятность, что файл был записан злонамеренно. Хотя наиболее очевидная форма несанкционированного изменения — на локальном диске, есть также множество способов сделать это, когда файл включается удалённо, как большая часть JavaScript в Интернете. Атакующий может нарушить вашу безопасность, нарушив безопасность чужого сайта. Они также могут (из-за плохих привилегий, подмены или повышения привилегий) изменять файлы, которыми владеете вы. Наконец, они могут изменять ссылки или перенаправления различных видов. Ссылки часто остаются вне проверок целостности. Есть несколько тонкий вариант этого, когда между вещами, которые вы контролируете (например, сервер), и вещами, которые вы не контролируете (например, веб-браузер на другом конце Интернета), есть кэши. Например, атаки отравления кэша вставляют данные в веб-кэши через плохие средства контроля безопасности в кэшах (OWASP, 2009).

Несанкционированное изменение памяти

Атакующие могут изменять ваш код, если они работают на том же уровне привилегий. В этот момент защита сложна. Если ваш API обрабатывает данные по ссылке (шаблон, часто выбираемый для скорости), то атакующий может изменить их после того, как вы выполните проверки безопасности.

Несанкционированное изменение сети

Сетевое несанкционированное изменение часто включает различные трюки, чтобы привести данные на машину атакующего, где он пересылает часть данных нетронутыми, а часть — изменёнными. Однако трюки, чтобы привести вам данные, не всегда нужны; с радиоинтерфейсами вроде WiFi и Bluetooth всё больше данных течёт по воздуху. Многие сетевые протоколы были разработаны с предположением, что вам нужно специальное оборудование для создания или чтения произвольных пакетов. Требование специального оборудования было защитой от несанкционированного изменения (и часто подмены). Рост программно-определяемого радио (SDR) молча аннулировал необходимость в специальном оборудовании. Теперь легко купить недорогой SDR-блок, который можно запрограммировать на несанкционированное изменение беспроводных протоколов.

Угрозы репудиации

Репудиация — это заявление, что вы чего-то не делали или не несли ответственности за то, что произошло. Люди могут репудиировать честно или обманно. Учитывая растущие знания, часто необходимые для понимания сложного мира, те, кто честно репудиирует,

могут действительно выявлять проблемы в вашем пользовательском опыте или архитектуре сервисов. Угрозы репудиации немного отличаются от других угроз безопасности, поскольку они часто появляются на бизнес-уровне. (То есть выше сетевого уровня, такого как TCP/IP, выше прикладного уровня, такого как HTTP/HTML, и там, где реализуется бизнес-логика покупки продуктов.)

Угрозы репудиации также связаны с вашей системой логирования и процессом. Если у вас нет логов, вы не храните логи или не можете анализировать логи, угрозы репудиации трудно оспорить. Есть также класс атак, в которых атакующие помещают данные в логи, чтобы сделать анализ логов сложным. Например, если вы отображаете свои логи в HTML, а атакующий отправляет </tr> или </html>, ваше отображение логов должно трактовать их как данные, а не как код. Больше угроз репудиации показано в таблице 3-4.

Таблица 3-4: Угрозы репудиации

УГРОЗА	ПРИМЕРЫ	ЧТО ДЕЛАЕТ АТАКУЮЩИЙ	ПРИМЕЧАНИЯ
Репудиация действия	Заявляет, что не кликал. Может быть, он действительно не кликал. Заявляет, что не получил. Получение может быть странным: означает ли загрузка почты вашим телефоном, что вы её прочитали? Предзагрузил ли сетевой прокси изображения? Оставил ли кто-то посылку на крыльце? Заявляет, что стал жертвой мошенничества. Использует чужую учётную запись. Использует чужой платёжный инструмент без авторизации		
Атака на логи	Замечает, что у вас нет логов. Помещает атаки в логи, чтобы запутать логи, код чтения логов или человека, читающего логи		

Атака на логи

Опять же, если у вас нет логов, вы не храните логи или не можете анализировать логи, действия по репудиации трудно оспорить. Поэтому, если вы не логируете, вам, вероятно, нужно начать. Если у вас нет централизации логов или возможностей анализа, вам, вероятно, это тоже нужно. Если вы не определяете должным образом, что будете логировать, атакующий может сломать вашу систему анализа логов. Может быть сложно

проработать слои производства и анализа логов, чтобы обеспечить надёжность, но если вы этого не сделаете, легко пропустить атаки через трещины или несоответствия.

Репудиация действия

Когда вы обсуждаете репудиацию, полезно обсуждать «кого-то», а не «атакующего». Вы хотите делать это потому, что те, кто репудиирует, часто на самом деле не атакующие, а люди, которых подвели технология или процесс. Может быть, они действительно не кликали (или не воспринимали, что кликнули). Может быть, спам-фильтр действительно съел это сообщение. Может быть, UPS не доставил, или, может быть, UPS доставил, оставив посылку на крыльце. Может быть, кто-то заявляет, что стал жертвой мошенничества, когда на самом деле это не так (или, может быть, кто-то другой в домохозяйстве использовал его кредитную карту, с его ведома или без). Хорошие технологические системы, которые и аутентифицируют, и хорошо логируют, могут облегчить处理 проблем репудиации.

Угрозы раскрытия информации

Раскрытие информации — это позволение людям видеть информацию, которую они не авторизованы видеть. Некоторые угрозы раскрытия информации показаны в таблице 3-5.

Таблица 3-5: Угрозы раскрытия информации

УГРОЗА	ПРИМЕРЫ	ЧТО ДЕЛАЕТ АТАКУЮЩИЙ	ПРИМЕЧАНИЯ
Раскрытие информации против процесса	Извлекает секреты из сообщений об ошибках. Читает сообщения об ошибках от имени пользователя/пароля до целых таблиц базы данных. Извлекает секреты машины из случаев ошибок. Может сделать защиту от повреждения памяти, такую как ASLR, гораздо менее полезной		
Раскрытие информации против хранилищ данных	Извлекает деловые/личные секреты из случаев ошибок. Пользуется неуместными или отсутствующими ACL. Пользуется плохими разрешениями базы данных. Находит файлы, защищённые неясностью. Находит криптоключи на диске (или в		

УГРОЗА	ПРИМЕРЫ	ЧТО ДЕЛАЕТ АТАКУЮЩИЙ	ПРИМЕЧАНИЯ
	памяти). Видит интересную информацию в именах файлов. Читает файлы, когда они проходят по сети. Получает данные из логов или временных файлов. Получает данные из swap или другого временного хранилища. Извлекает данные, получив устройство, изменив ОС		
Раскрытие информации против потока данных	Читает данные в сети. Перенаправляет трафик, чтобы включить чтение данных в сети. Узнаёт секреты, анализируя трафик. Узнаёт, кто с кем говорит, наблюдая за DNS. Узнаёт, кто с кем говорит, через раскрытие информации в социальных сетях		

Раскрытие информации из процесса

Многие случаи, когда процесс раскрывает информацию, — это те, которые информируют дальнейшие атаки. Процесс может делать это, утекая адреса памяти, извлекая секреты из сообщений об ошибках или извлекая детали дизайна из сообщений об ошибках. Утечка адресов памяти может помочь обойти ASLR и подобные защиты. Утечка секретов может включать строки подключения к базе данных или пароли. Утечка деталей дизайна может означать раскрытие анти-мошеннических правил вроде «ваша учётная запись слишком новая, чтобы заказать бриллиантовое кольцо».

Раскрытие информации из хранилища данных

Поскольку хранилища данных, ну, хранят данные, есть множество способов, которыми они могут их утечь. Первый набор причин — это failures правильно использовать механизмы безопасности. Неустановка разрешений должным образом или надежда, что никто не найдёт неясный файл, — распространённые способы, которыми люди не используют механизмы безопасности. Криптографические ключи — особый случай, когда раскрытие информации позволяет дополнительные атаки. Файлы, читаемые из хранилища данных по сети, часто читаемы, когда они проходят по сети.

Дополнительная атака, часто упускаемая, — данные в именах файлов. Если у вас есть каталог с именем «May 2013 layoffs», само имя файла «Termination Letter for Alice.docx» раскрывает важную информацию.

Есть также группа атак, в которых программа испускает информацию в операционную среду. Логи, временные файлы, swap или другие места могут содержать данные. Обычно ОС защищает данные в swap, но для таких вещей, как криптоключи, вы должны использовать средства ОС, чтобы предотвратить их выгрузку в swap.

Наконец, есть класс атак, в которых данные извлекаются из устройства с использованием операционной системы под контролем атакующего. Наиболее часто (в 2013 году) эти атаки затрагивают USB-ключи, но они также применимы к CD, лентам резервного копирования, жёстким дискам или украденным ноутбукам или серверам. Жёсткие диски часто выводятся из эксплуатации без полного удаления данных. (Вы можете address необходимость удаления данных с жёстких дисков, купив измельчитель жёстких дисков или машину для разбивания, и поскольку такие машины awesome, почему бы и нет?)

Раскрытие информации из потока данных

Потоки данных особенно подвержены атакам раскрытия информации, когда информация течёт по сети. Однако потоки данных на одной машине всё ещё могут быть атакованы, особенно когда машина используется совместно облачными со-арендаторами или множеством взаимно недоверяющих пользователей вычислительного сервера. Помимо простого чтения данных в сети, атакующие могут перенаправлять трафик к себе (часто путём подмены некоторого протокола управления сетью), чтобы видеть его, когда они не на нормальном пути. Также возможно получить информацию, даже когда сетевой трафик зашифрован. Есть множество способов узнать секреты о том, кто с кем говорит, включая наблюдение за DNS, активность друзей на сайте вроде LinkedIn или другие формы анализа социальных сетей.

ПРИМЕЧАНИЕ. Знатоки безопасности могут задаваться вопросом, будут ли упомянуты атаки по сторонним каналам и скрытые каналы. Эти атаки могут быть весёлыми в работе (и сторонние каналы немного рассматриваются в главе 16 «Threats to Cryptosystems»), но они не актуальны, пока вы не смягчили проблемы, рассмотренные здесь.

Угрозы отказа в обслуживании

Атаки отказа в обслуживании поглощают ресурс, необходимый для предоставления сервиса. Примеры описаны в таблице 3-6.

Таблица 3-6: Угрозы отказа в обслуживании

УГРОЗА	ПРИМЕРЫ	ЧТО ДЕЛАЕТ АТАКУЮЩИЙ	ПРИМЕЧАНИЯ
Отказ в обслуживании против процесса	Поглощает память (RAM или диск). Поглощает		

УГРОЗА	ПРИМЕРЫ	ЧТО ДЕЛАЕТ АТАКУЮЩИЙ	ПРИМЕЧАНИЯ
	CPU. Использует процесс как усилитель		
Отказ в обслуживании против хранилища данных	Заполняет хранилище данных. Делает достаточно запросов, чтобы замедлить систему		
Отказ в обслуживании против потока данных	Потребляет сетевые ресурсы		

Атаки отказа в обслуживании можно разделить на те, которые работают, пока атакующий атакует (скажем, заполняет пропускную способность), и те, которые сохраняются. Постоянные атаки могут оставаться в силе до перезагрузки (например, while (1) {fork();}) или даже после перезагрузки (например, заполнение диска). Атаки отказа в обслуживании также можно разделить на усиленные и неусиленные. Усиленные атаки — это те, где небольшие усилия атакующего приводят к большому воздействию. Пример использовал бы старую службу unix charger, цель которой — генерировать полу-случайную схему символов для тестирования. Атакующий мог подменить один пакет с порта charger на машине A на порт charger на машине B. Веселье продолжается, пока кто-то не выдернет сетевой кабель.

Угрозы повышения привилегий

Повышение привилегий — это позволение кому-то делать то, на что он не авторизован — например, позволение обычному пользователю выполнять код как admin или позволение удалённому человеку без каких-либо привилегий запускать код. Два важных способа повысить привилегии включают повреждение процесса и обход проверок авторизации. Примеры показаны в таблице 3-7.

Таблица 3-7: Угрозы повышения привилегий

УГРОЗА	ПРИМЕРЫ	ЧТО ДЕЛАЕТ АТАКУЮЩИЙ	ПРИМЕЧАНИЯ
Повышение привилегий против	Отправляет входные данные, которые код не обрабатывает		

УГРОЗА	ПРИМЕРЫ	ЧТО ДЕЛАЕТ АТАКУЮЩИЙ	ПРИМЕЧАНИЯ
процесса путём повреждения процесса	должным образом. Эти ошибки очень распространены и обычно имеют высокое воздействие. Получает доступ к чтению или записи памяти ненадлежащим образом. Запись в память — (надеюсь, очевидно) плохо, но чтение памяти может позволить дальнейшие атаки		
Повышение через пропущенные проверки авторизации			
Повышение через багованные проверки авторизации	Централизация таких проверок облегчает управление багами		
Повышение через несанкционированное изменение данных	Изменяет биты на диске, чтобы делать вещи, отличные от того, что намеревался авторизованный пользователь		

Повышение привилегий путём повреждения процесса

Повреждение процесса включает вещи вроде разрушения стека, эксплуатации данных в куче и целого разнообразия техник эксплуатации. Воздействие этих техник в том, что атакующий получает влияние или контроль над потоком управления программы. Важно понимать, что эти эксплойты не ограничены поверхностью атаки. Первый код, до которого могут добраться данные атакующего, конечно, важная цель. Обычно этот код может проверять данные только для ограниченного поднабора целей. Важно проследить потоки данных дальше, чтобы увидеть, где ещё может произойти повышение привилегий. Есть несколько необычный случай, когда программа полагается на разделяемую память и

выполняет вещи из неё, что является тривиальным путём для повышения, если всё, что имеет разрешения на эту разделяемую память, не работает на том же уровне привилегий.

Повышение привилегий через отказы авторизации

Есть также набор способов повысить привилегии через отказы авторизации. Простейший отказ — не проверять авторизацию на каждом пути. Более сложный для атакующего — пользоваться багованными проверками авторизации. Наконец, если программа полагается на другие программы, конфигурационные файлы или наборы данных как на заслуживающие доверия, важно убедиться, что разрешения установлены так, что каждая из этих зависимостей должным образом защищена.

Расширенный пример: угрозы STRIDE против Acme-DB

Этот расширенный пример обсуждает, как угрозы STRIDE могли бы проявиться против базы данных Acme/SQL, описанной в главах 1 «Dive In and Threat Model!» и 2 «Strategies for Threat Modeling» и показанной на рисунке 2-1. Сначала вы рассмотрите эти угрозы по категориям STRIDE, а затем изучите тот же набор в соответствии с тем, кто может их устранить.

Подмена

- Веб-клиент мог бы попытаться войти со случайными учётными данными или украденными учётными данными, как и SQL-клиент.
- Если вы предполагаете, что SQL-клиент — тот, который вы написали, и позволяете ему принимать решения по безопасности, то подменённый (или изменённый) клиент мог бы обойти проверки безопасности.
- Веб-клиент мог бы подключиться к ложному (подменённому) фронтенду и в итоге раскрыть учётные данные.
- Программа могла бы притвориться базой данных или программой анализа логов и попытаться прочитать данные из различных хранилищ данных.

Если сеть между основными процессами или между процессами и базами данных является общей, она может стать перегруженной.

Повышение привилегий

- Клиенты, как веб-, так и SQL, могли бы попытаться выполнить запросы, на которые они не авторизованы.
- Если клиент обеспечивает безопасность, то любой, кто изменит их клиент или его сетевой поток, сможет выполнять запросы по своему выбору.
- Если база данных способна выполнять произвольные команды, то эта возможность доступна клиентам.
- Программа анализа логов (или что-то, притворяющееся программой анализа логов) может иметь возможность выполнять произвольные команды или запросы.

ПРИМЕЧАНИЕ. Программа анализа логов может считаться доверенной, но она нарисована вне границ доверия. Так что либо мышление, либо диаграмма (на рисунке 2-1) неверны.

Если кластер DB подключён к корпоративной службе каталогов и не предпринято никаких действий для ограничения того, кто может входить на серверы базы данных (или файловые серверы), то любой в корпоративном каталоге, включая, возможно, сотрудников, подрядчиков, лаборатории сборки и партнёров, может вносить изменения в этих системах.

ПРИМЕЧАНИЕ. Предыдущие списки в этом расширенном примере предназначены быть иллюстративными; другие угрозы могут существовать.

Также возможно рассмотреть эти угрозы в соответствии с человеком или командой, которые должны их устранить, разделив между Acme и её клиентами. Как показано в таблице 3-8, это иллюстрирует естественное пересечение угрозы и меры защиты, предвещая часть III «Managing and Addressing Threats» о том, как смягчать угрозы. Это также начинает перечислять вещи, которые не являются требованиями для Acme/SQL. Эти не-требования должны быть задокументированы и предоставлены клиентам, как описано в главе 12. В этой таблице вы видите всё более и более действенные угрозы. Как разработчик или системный администратор, вы можете начать видеть, как обрабатывать такие проблемы. Соблазнительно начать address угрозы в самой таблице, и естественным расширением таблицы был бы набор способов для каждого действующего лица address применимые к нему угрозы.

Таблица 3-8: Устранение угроз в соответствии с тем, кто их обрабатывает

УГРОЗА	СЛУЧАИ, КОТОРЫЕ ДОЛЖНА ОБРАБАТЫВАТЬ ACME	СЛУЧАИ, КОТ ДОЛЖНЫ ОБРАБАТЫВА ОТДЕЛЫ
Подмена	Веб-/SQL-/другой клиент брутфорсит логины. Веб-клиент. DBA (человек). SQL-клиент. Пользователи БД. DBA (человек)	
Несанкционированное изменение	Данные. Front end(s). Управление. База данных. Логи. DB admin	
Репудиация	Логи (анализ логов должен быть защищён). Логи (анализ логов должен быть защищён). Определённые действия от веб- и SQL-клиентов потребуют тщательного логирования. Если DBA не полностью доверенные, может потребоваться система в другом домене привилегий для логирования всех команд. Определённые действия от DBA потребуют тщательного логирования	

УГРОЗА	СЛУЧАИ, КОТОРЫЕ ДОЛЖНА ОБРАБАТЫВАТЬ ACME	СЛУЧАИ, КОТ ДОЛЖНЫ ОБРАБАТЫВА ОТДЕЛЫ
Раскрытие информации	Данные, управление и логи должны быть защищены. ACL и группы безопасности должны управляться. Front ends должны реализовывать контроль доступа. Резервные копии должны быть защищены. Только front ends должны иметь доступ к данным. Система должна быть развёрнута с достаточными ресурсами	
Отказ в обслуживании	Front ends должны быть спроектированы для минимизации рисков DoS. Система должна быть развёрнута с достаточными ресурсами	
Повышение привилегий	Доверие клиенту. БД должна поддерживать подготовленные выражения, чтобы сделать инъекцию сложнее. В установке по умолчанию не должно быть инструментов «выполнить эту команду». Не должно быть способа по умолчанию запускать команды на сервере, и вызовы вроде exec() и system() должны быть разрешены и настраиваемы, если они существуют. Неподобающее доверие клиентам, написанным локально. Настройте БД соответствующим образом	

Варианты STRIDE

STRIDE может быть очень полезной мнемоникой при поиске угроз, но она не идеальна. В этом разделе вы узнаете о вариантах STRIDE, которые могут помочь address некоторые из её слабостей.

STRIDE-per-Element

STRIDE-per-element делает STRIDE более предписывающей, отмечая, что определённые угрозы более распространены с определёнными элементами диаграммы. Например, хранилище данных вряд ли подменит другое хранилище данных (хотя выполняющийся код может быть запутан в том, к какому хранилищу данных он обращается). Сосредоточившись на наборе угроз против каждого элемента, этот подход облегчает поиск угроз. Например, Microsoft использует таблицу 3-9 как ключевую часть своего обучения моделированию угроз в Security Development Lifecycle.

Таблица 3-9: STRIDE-per-Element

STRIDE	Внешняя сущность	Процесс	Поток данных	Хранилище данных
Spoofing	×	×		
Tampering		×	×	×
Repudiation		×		?
Information Disclosure		×	×	×
Denial of Service		×	×	×
Elevation of Privilege		×		

Применяя эту таблицу, вы можете сосредоточить анализ угроз на том, как атакующий мог бы изменить, прочитать данные из или предотвратить доступ к потоку данных. Например, если данные текут по сети, такой как Ethernet, тривиально для кого-то, подключённого к тому же Ethernet, прочитать всё содержимое, изменить его или отправить поток пакетов, вызывающий тайм-аут TCP. Вы могли бы возразить, что у вас есть некоторая форма сегментации сети, и это может достаточно смягчить угрозы для вас. Знак вопроса под репудиацией указывает, что хранилища данных-логов участвуют в address репудиации, и иногда логи подвергаются специальной атаке, чтобы позволить атаки репудиации.

Угроза — это угроза элементу, указанному в таблице 3-9. Каждый элемент — жертва, а не преступник. Поэтому, если вы изменяете хранилище данных, угроза — хранилищу данных и данным внутри. Если вы подменяете способом, влияющим на процесс, то процесс — жертва. Так, подмена путём несанкционированного изменения сети — это на самом деле подмена конечной точки, независимо от технических деталей. Другими словами, другая конечная точка (или конечные точки) запутаны относительно того, что находится на другом конце соединения. Таблица фокусируется на подмене процесса, а не на подмене потока данных. Конечно, если вы случайно найдёте подмену, глядя на поток данных, очевидно, вы должны записать угрозу, чтобы address её, а не беспокоиться о том, что это за тип угрозы. STRIDE-per-element имеет преимущество быть предписывающей, помогая вам определить, что искать где, не будучи контрольным списком формы «веб-компонент: XSS, XSRF...». В умелых руках она может использоваться для поиска новых типов слабостей в компонентах. В менее умелых руках она всё ещё может находить многие распространённые проблемы.

STRIDE-per-element имеет два недостатка. Во-первых, похожие проблемы, как правило, всплывают repeatedly в данной модели угроз; во-вторых, таблица может не представлять

ваши проблемы. Фактически, таблица 3-9 несколько специфична для Microsoft. Самое лёгкое место, чтобы увидеть это — «раскрытие информации внешней сущностью», что является хорошим описанием некоторых проблем приватности. (Это ни в коем случае не полное описание приватности.) Однако таблица не указывает, что это может быть проблемой. Это потому, что у Microsoft есть отдельный набор процессов для анализа проблем приватности. Эти процессы приватности находятся вне пространства моделирования угроз безопасности. Поэтому, если вы собираетесь принять этот подход, стоит проанализировать, охватывает ли таблица набор проблем, которые вас волнуют, и если нет, создайте версию, подходящую для вашего сценария. Другое место, где вы можете увидеть специфичность, — то, что многие люди хотят обсуждать подмену потоков данных. Должна ли она быть частью STRIDE-per-element? Действие подмены — это подмена конечной точки, но это описание может помочь некоторым людям искать эти угрозы. Также обратите внимание, что чем больше «x» вы добавляете, тем ближе вы подходите к «рассмотрите STRIDE для каждого элемента диаграммы». Редакторы спрашивают, хорошо это или плохо, и это хороший вопрос. Если вы хотите быть всеобъемлющим, это полезно; если вы хотите сосредоточиться на наиболее вероятных проблемах, однако, это, вероятно, будет отвлекать.

Итак, каковы критерии выхода для STRIDE-per-element? Когда у вас есть угроза на каждую галочку в таблице STRIDE-per-element, вы делаете reasonably хорошо. Если вы обойдёте по кругу и рассмотрите угрозы против ваших мер защиты (или способы их обхода), вы будете делать довольно хорошо.

STRIDE-per-Interaction

STRIDE-per-element — это упрощённый подход к выявлению угроз, разработанный, чтобы быть легко понятным новичку. Однако в реальности угрозы не появляются в вакууме. Они появляются во взаимодействиях системы. STRIDE-per-interaction — это подход к перечислению угроз, который рассматривает кортежи (origin, destination, interaction) и перечисляет угрозы против них. Изначально другой целью этого подхода было уменьшить количество вещей, которые модельщику пришлось бы рассматривать, но это не сработало, как планировалось. STRIDE-per-interaction приводит к тому же количеству угроз, что и STRIDE-per-element, но угрозы могут быть легче поняты с этим подходом. Этот подход был разработан Ларри Остерманом и Дугласом Макайвером, оба из Microsoft. Подход STRIDE-per-interaction показан в таблицах 3-10 и 3-11. Обе ссылаются на два процесса, Contoso.exe и Fabrikam.dll. Таблица 3-10 показывает, какие угрозы применяются к каждому взаимодействию, а таблица 3-11 показывает пример STRIDE per interaction, применённый к рисунку 3-1. Отношения и границы доверия, использованные для названных элементов в обеих таблицах, показаны на рисунке 3-1.

Браузер → Команды → Contoso.exe → Write → database
Contoso.exe → Ответы → Браузер
Contoso.exe → Create(widgets) → Fabrikam.dll
Fabrikam.dll → widgets → Contoso.exe

Рисунок 3-1: Система, на которую ссылается таблица 3-10

В таблице 3-10 столбцы таблицы следующие:

- Номер для ссылки на строку (например, «Глядя на строку 2, давайте поищем угрозы подмены и раскрытия информации».)
- Основной элемент, который вы рассматриваете
- Взаимодействия, которые имеет этот элемент
- Угрозы STRIDE, применимые к взаимодействию

Таблица 3-10: STRIDE-per-Interaction: Применимость угроз

#	ЭЛЕМЕНТ	ВЗАИМОДЕЙСТВИЕ	S	T	R	I	D	E
1	Процесс (Contoso)	Процесс имеет исходящий поток данных к хранилищу данных	×	×				
2	Процесс отправляет вывод другому процессу	×	×	×	×	×		
3	Процесс отправляет вывод внешнему интерактору (код)	×	×	×	×			
4	Процесс отправляет вывод внешнему интерактору (человек)	×						
5	Процесс имеет входящий поток данных из хранилища данных	×	×	×	×			
6	Процесс имеет входящий поток данных из процесса	×	×	×	×	×		
7	Процесс имеет входящий поток данных из	×	×	×				

#	ЭЛЕМЕНТ	ВЗАИМОДЕЙСТВИЕ	S	T	R	I	D	E
	внешнего интерактора							
8	Поток данных (команды/ответы)	Пересекает границу машины	×	×	×			
9	Хранилище данных (база данных)	Процесс имеет исходящий поток данных к хранилищу данных	×	×	×	×		
10	Процесс имеет входящий поток данных из хранилища данных	×	×	×				
11	Внешний интерактор (браузер)	Внешний интерактор передаёт ввод процессу	×	×	×			
12	Внешний интерактор получает ввод от процесса	×						

Таблица 3-11: STRIDE-per-Interaction: Пример

ЭЛЕМЕНТ	ВЗАИМОДЕЙСТВИЕ	S	T	R	I	D	E
5	Процесс имеет входящий поток данных из хранили	«База данных» подменяется, и Contoso	Состояние процесса Contoso повреждено	Contoso отрицает получение данных	Contoso получает данные от	Contoso падает/останавливается из-за взаимодействия с Fabrikam.	Contoso падает/останавливается из-за взаимодействия с браузером.

ЭЛЕМЕНТ	ВЗАИМОДЕЙСТВИЕ	S	T	R	I	D	E
	ща данных	читает неверные данные.	о данными, полученными из хранилища данных.	ных от Fabrikam.	Fabrikam.		
6	Процесс имеет входящий поток данных из процесса	Contoso верит, что получает данные от Fabrikam.					
7	Процесс имеет входящий поток данных из внешнего интерактора	Contoso верит, что получает данные от браузера, когда на самом деле это случа					

ЭЛЕМЕНТ	ВЗАИМОДЕЙСТВИЕ	S	T	R	I	D	E
		йный атакующий.					

Когда у вас есть угроза на каждую галочку в таблице STRIDE-per-interaction, вы делаете reasonably хорошо. Если вы обойдёте по кругу и рассмотрите угрозы против ваших мер защиты (или способы их обхода), вы будете делать довольно хорошо.

STRIDE-per-interaction слишком сложен для использования без справочной таблицы под рукой. (Напротив, STRIDE — лёгкая мнемоника, а STRIDE-per-element достаточно прост, чтобы таблицу можно было запомнить или напечатать на карточке для бумажника.)

DESIST

DESIST — это вариант STRIDE, созданный Гуннаром Петерсоном. DESIST расшифровывается как Dispute, Elevation of privilege, Spoofing, Information disclosure, Service denial и Tampering. (Dispute заменяет repudiation менее причудливым словом, а Service denial заменяет Denial of Service, чтобы акроним работал.) Начиная с нуля, возможно, имело бы смысл использовать DESIST вместо STRIDE, но после более чем десятилетия STRIDE было бы дорого вытеснить её в Microsoft. (Генеральный директор Scorpion Software Дана Эпп отметил, что акронимы с повторяющимися буквами могут быть сложными, что является плюсом для STRIDE.) Поэтому STRIDE-per-element, а не DESIST-per-element, существует как норма. В любом случае всегда полезно иметь мнемоники, помогающие людям искать угрозы.

Критерии выхода

Есть три способа судить, закончили ли вы поиск угроз с помощью STRIDE. Самый лёгкий — посмотреть, есть ли у вас угроза каждого типа в STRIDE. Немного сложнее — убедиться, что у вас есть одна угроза на каждый элемент диаграммы. Однако оба этих критерия будут достигнуты до того, как вы найдёте все угрозы. Для большей всеобъемлемости используйте STRIDE-per-element и убедитесь, что у вас есть одна угроза на каждую галочку.

Невыполнение этих критериев скажет вам, что вы не закончили, но их выполнение не является гарантией полноты.

Резюме

STRIDE — полезная мнемоника для поиска угроз против всех видов технологических систем. STRIDE более полезна с репертуаром более детальных угроз, на который можно опереться. Таблицы угроз могут обеспечить это для тех, кто новичок в безопасности, или действовать как справочный материал для экспертов по безопасности (функция, которую также выполняет Приложение B «Threat Trees»). Есть варианты STRIDE, которые пытаются

добавить фокус и внимание. STRIDE-per-element очень полезна для этой цели и может быть настроена под ваши нужды. STRIDE-per-interaction обеспечивает больший фокус, но требует шпаргалки (или, возможно, ПО) для использования. Если бы эксперты по моделированию угроз начинали заново, возможно, DESIST помогла бы нам сделать лучший... прогресс в поиске угроз.